Lab03S03 · Laboratório de Desenvolvimento de Software

Sistema de Moeda Estudantil

Arquitetura do sistema e camada de persistência

React · Express · Prisma · PostgreSQL



Natã Torres Dias · 10/11/2026

Quem usa o sistema e o que cada um faz

Professor

Recebe 1.000 moedas por semestre (acumuláveis) e envia aos seus alunos com um motivo obrigatório.

Aluno

Cadastra-se, recebe moedas, consulta o extrato e troca o saldo por vantagens.

Empresa parceira

Cadastra vantagens com descrição, foto e custo em moedas, e confere os cupons de troca.

> Todos acessam com login e senha. Instituições e professores já vêm pré-cadastrados pela parceria.

Tecnologias que escolhi e por quê

Camada	Tecnologia	Por quê
View	React 19 + Vite, React Router, Axios	SPA com componentes reutilizáveis
Controller / Service	Node.js + Express 5	API REST simples, async nativo
Model	Prisma 6 (ORM) + padrão DAO	Esquema versionado, consultas tipadas
Banco de dados	PostgreSQL 16 (Docker)	Relacional, enums e restrições
Segurança	JWT + bcrypt	Sessão sem estado, senha com hash

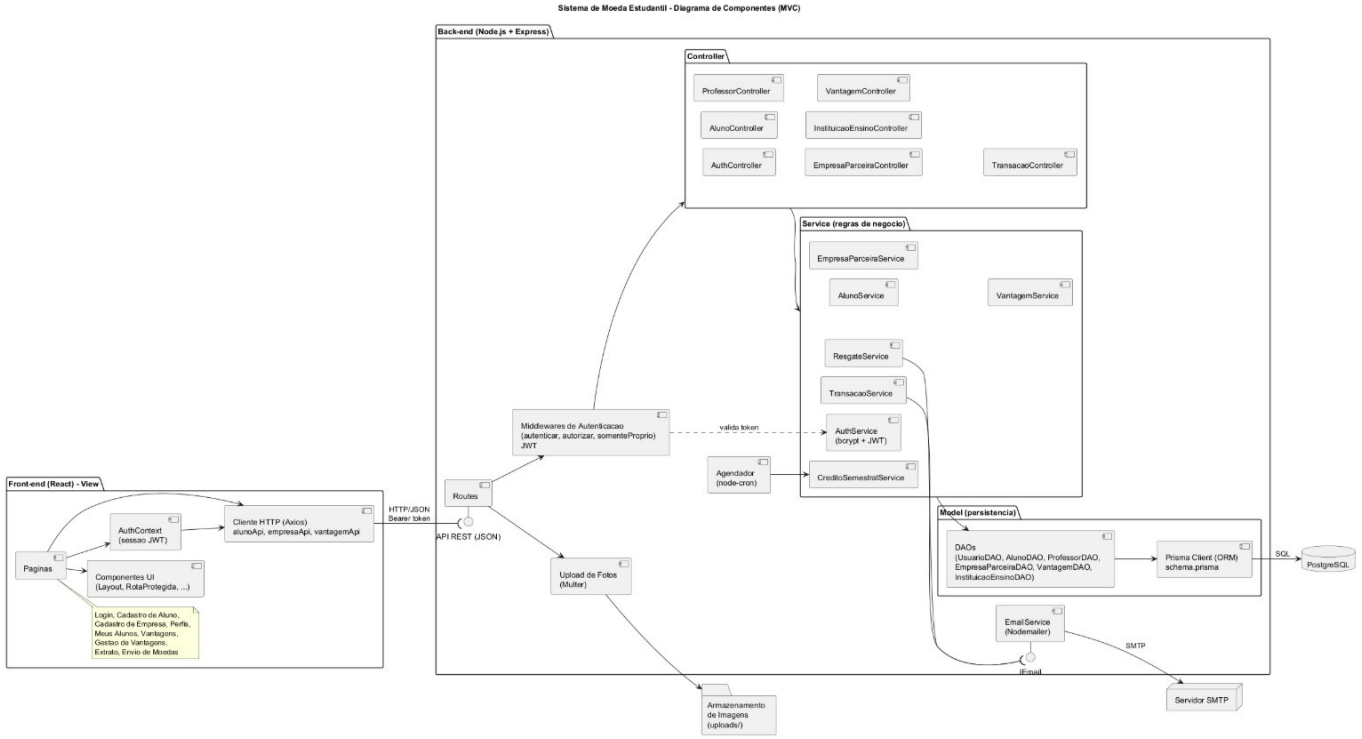
Como dividi o código: MVC com camada de serviço



Front e back separados: o React só conversa com a API por JSON, usando o Axios.

Regra: controller não acessa o banco, e só os DAOs importam o Prisma.

Diagrama de componentes



Passo a passo: a Maria edita o próprio perfil

`PUT /api/alunos/7`

1	**AlunoForm**	Axios envia o JSON com o token JWT no cabeçalho Authorization
2	**Middlewares**	autenticar (token válido?) → autorizar('ALUNO') → somenteProprio (id do token = 7?)
3	**AlunoController**	Lê o id, chama o service e responde 200 com o aluno em JSON
4	**AlunoService**	Valida CPF, email, endereço e instituição; gera o hash da nova senha
5	**AlunoDAO**	Uma operação aninhada atualiza usuario, aluno e endereco
6	**Prisma → PostgreSQL**	SQL numa transação; CPF duplicado volta como 409

03 · camada de persistência

Como o sistema conversa com o banco de dados?

Usei o Prisma (um ORM) e, por cima dele, um DAO para cada entidade. A seguir: os DAOs, o modelo ER, como o banco é criado e o que acontece quando ele recusa um dado.

Por que ORM e DAO juntos

O que o Prisma faz por nós

schema.prisma é a fonte da verdade do modelo

Migrações SQL geradas e versionadas no Git

Cliente tipado com relações aninhadas

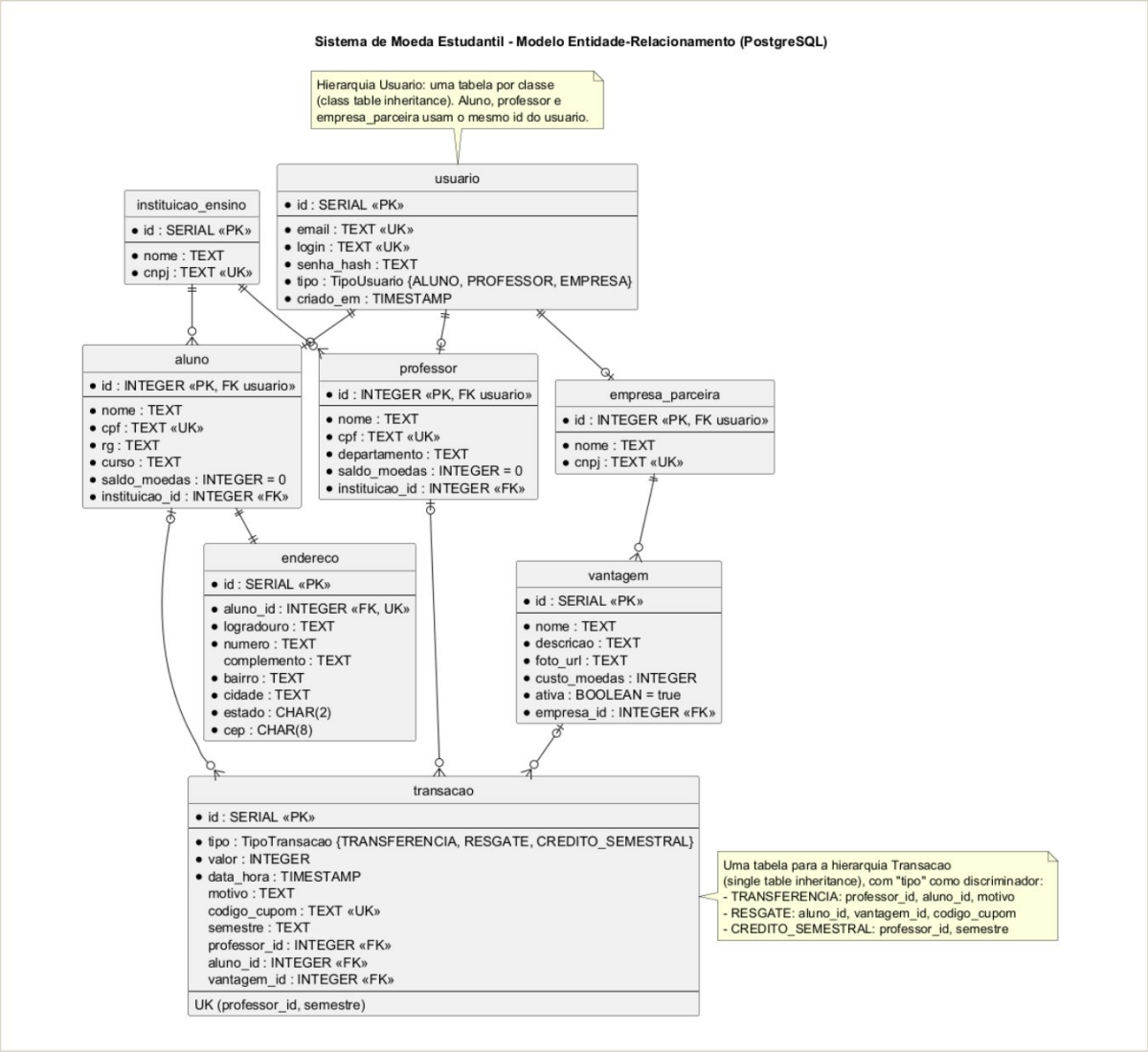
O que o DAO acrescenta

Um DAO por entidade: create, findAll, findById, update, delete

Só os DAOs importam o Prisma

Trocar de ORM ou de banco muda só esta camada

```
AlunoService → alunoDAO.create(dados) → prisma.usuario.create({ aluno: { create: { endereco: … } } })
```



03 · persistência · modelo ER

As 8 tabelas do banco

aluno, professor e empresa_parceira usam o id do usuario como chave primária e estrangeira

transacao guarda os três tipos de movimento de moedas

Únicos: email, login, CPF, CNPJ, código do cupom e (professor, semestre)

Como as heranças do diagrama viraram tabelas

`Usuario → Aluno | Professor | EmpresaParceira`

Uma tabela por classe

usuario guarda email, login e hash da senha; cada perfil tem sua tabela com o mesmo id.

> **Por quê:** perfis com atributos bem diferentes, e um único lugar para autenticar.

`Transacao → Transferência | Resgate | Crédito`

Uma tabela só

transacao tem a coluna tipo como discriminador e campos opcionais por tipo.

> **Por quê:** o extrato vira uma consulta só, ordenada por data.

Como o banco é criado e preenchido

1 Subir

`docker compose up -d`

PostgreSQL 16 num container, dados num volume

2 Criar

`npx prisma migrate deploy`

Aplica o migration.sql: tabelas, enums, chaves e índices únicos

3 Popular

`npx prisma db seed`

3 instituições, 2 professores com 1.000 moedas, contas de demo

4 Usar

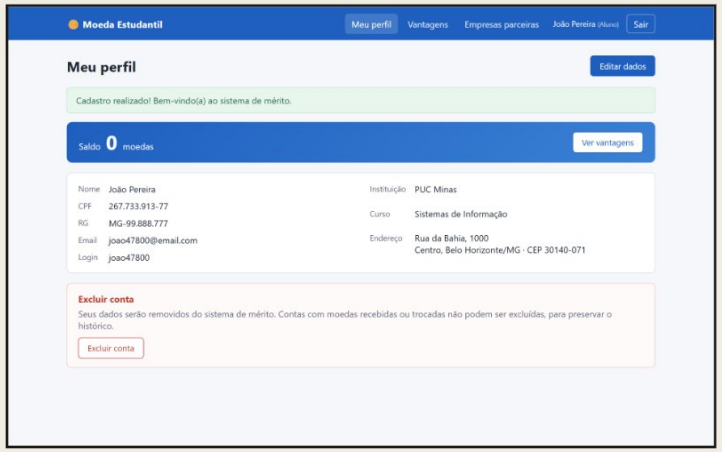
`npm run dev`

Cadastros pelas telas gravam via DAOs

Seed idempotente: usa upsert, pode rodar de novo sem duplicar nada.

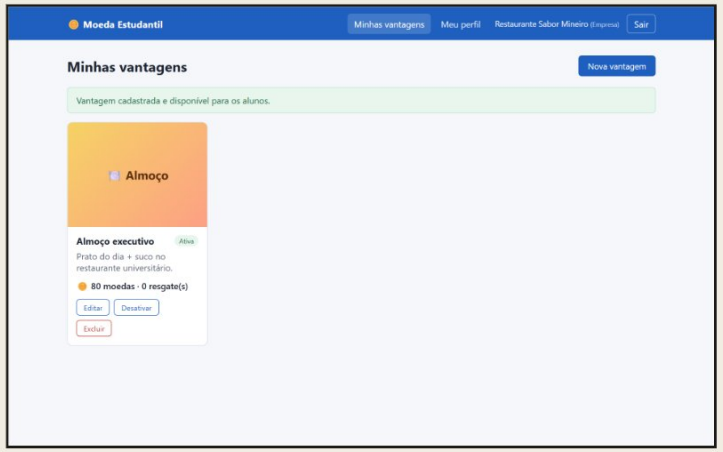
Banco de teste: npm test migra moeda_estudantil_test e o limpa antes de cada suíte.

As telas: versão final dos CRUDs



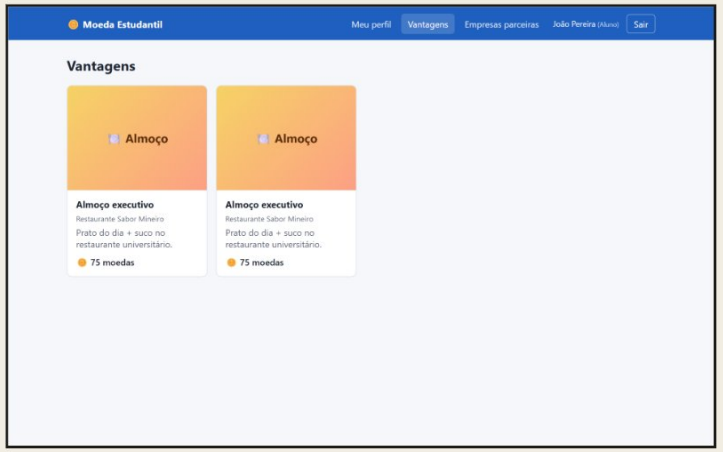
Aluno

Cadastro público, perfil com saldo, edição de dados e senha, exclusão da conta



Empresa parceira

Perfil e vantagens com foto, custo, ativação e exclusão



Catálogo

O aluno vê as vantagens ativas de todas as empresas

O que fica para as próximas sprints

Envio de moedas

TransacaoService com saldo suficiente e motivo obrigatório

Extrato

Uma consulta na tabela transacao, por aluno ou professor

Resgate com cupom

Código único e emails ao aluno e à empresa (Nodemailer)

Crédito semestral

Agendador (node-cron) soma 1.000 moedas por semestre

Obrigado! Perguntas?